

National University of Computer and Emerging Sciences, Lahore Campus

	Course Name:	Programming Fundamentals	Course Code:	CS
	Degree Program:	CS	Semester:	Fall 2019
	Exam Duration:	180 Minutes	Total Marks:	50
	Paper Date:	12-Dec-19	Weight	
	Section:	ALL	Page(s):	5
	Exam Type:	Final Exam		

Student : Name: **Muhammad Hassan Khalid** **Roll No.** **21L-5692** **Section:** **BDS -1A**

- Instruction/Notes:**
- Solve the exam on this question paper. Rough sheets can be used but will not be attached or marked.
 - In the programming problems, you are free to use the standard C++ library functions from iostream, cstring etc. where required.

Q1. [5 + 5] (a) Print the output of the following code:

<pre> void processNums(int nums[], int n) { int i=-1, j=0; for(;j<n; ++j){ if(nums[j]==1){ swap(nums[i+1],nums[j]); ++i; } } } </pre>	<pre> int main() { int nums[]={0, 1, 0, 0, 1, 1, 1, 0, 1, 0}; int n = 10; cout << "Original array: "; for (int i=0; i < n; i++) cout << nums[i] <<" "; processNums(nums, n); cout << "\nProcessed array: "; for (int i=0; i < n; i++) cout << nums[i] <<" "; return 0; } </pre>
Output: Original Array: 0 1 0 0 1 1 1 0 1 0 Processed array: 1 1 1 1 1 0 0 0 0 0	

(b) Print the output of the following code:

<pre> void UpdateStr(char str[]) { for (int i = 1; str[i] != '\0'; i++){ if (str[i] >= 'a' && str[i] <= 'z' && str[i-1] == ' ') str[i] = str[i] - 32; } } </pre>	
<pre> int main() { char str[] = "I am a student of programming!"; UpdateStr(str); cout << str << endl; } </pre>	
Output: I Am A Student Of Programming!	

Q2. [10 + 4]

(a) A black and white image is stored in a matrix *data* of dimensions $m \times n$. That is, it has m rows, n columns and $n \leq 1000$. Matrix *data* contains only 0s and 1s. 0 represents a black pixel and 1 represents white pixel. We are interested in counting the number of $k \times k$ blocks (sub-matrices) in *data* that have at least $k^2/2$ white pixels in them. We call such blocks “*mostly white blocks*”. Write a C++ function that returns the count of “*mostly white blocks*” in *data*.

You may use the function header: `int countMostlyWhite(int data[][1000], int m, int n, int k)`

For further clarity consider the following example: Here matrix *data* has dimensions $m = 5$, $n = 4$, and let's take $k = 3$. Take each 3×3 sub-matrix (this example has a total of 6 such matrices), and count its white pixels. Each “*mostly white block*” must have at least $3^2/2 = 4.5 \approx 4$ white pixels for $k = 3$. In the example there are 4 “*mostly white blocks*” of size 3×3 with their top-left corners at (0, 1), (1, 1) (2, 0) and (2, 1) respectively.

0	0	1	0
0	1	0	1
0	0	0	1
1	0	1	0
0	1	1	1

```
int countMostlyWhite(int data[][1000], int m, int n,
int k){
    int total=0;
    int value=(k*k)/2;
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            int sum=0;
            int c=i;
            int d=j;
            for(int a=i;a<k+c;a++){
                for(int b=j;b<k+d;b++){
                    if(((k+c)<=m)&&((k+d)<=n)){
                        if (data[a][b]==1){
                            sum++;
                        }
                    }
                }
            }
            if (sum>=value){
                total++;
            }
        }
    }
    return total;
}
```

(b) Suppose in part (a) above we also asked you to report the top left coordinates (row number and col number of the top left cell) of each and every *mostly white blocks* in *data*. Write down the changed function header that would be required to accomplish this task. Particularly, what new parameters may be required? Explain the purpose of these parameters n two lines.

```
int countMostlyWhite(int data[][1000], int m, int n, int k,int y[],int z[])
```

I will make two arrays in one array i will store the row of that co-ordinate
and in the other array i will store the column of that co-ordinate

Q3. [6] Short Questions

(a) Write a cout statement that prints **H"e"llo** on the screen.

```
cout<< "H"<<(char)34<<"e"<<(char)34<<"llo";
```

(b) Properly indent the following code.

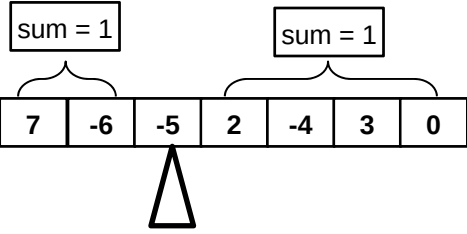
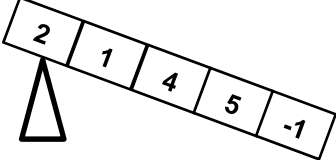
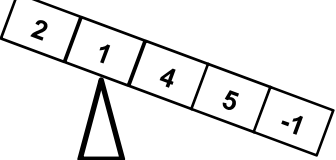
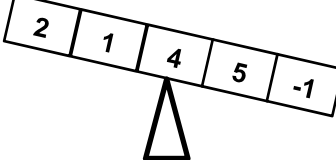
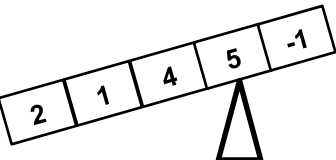
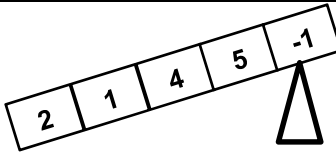
```
int a=1,b=2;if(a<b)cout<<"first"; else cout<<"second";
```

```
int a=1,b=2;
if(a<b)
cout<<"first";
else
cout<<"second";
```

(c) What is the binary equivalent for $(1010)_{10}$ (number given in decimal system)?

```
1111110010
```

Q4. [10] Write a C++ function called **findEquilibrium** that accepts an integer array *A* of size *n* and finds the *equilibrium index* of the array if it exists. An index *k* of an array is said to be an equilibrium index if the sum of all the elements on the left of *k* is equal to the sum of all the elements on the right of *k*. Precisely, *k* is an equilibrium index of *A* if and only if $\sum_{i=0}^{k-1} A[i] = \sum_{i=k+1}^{n-1} A[i]$. For clarity, consider this example below. If the index is not found, the function returns -1.

Equilibrium Index Found	Equilibrium Index not Found
	 [0] is not equilibrium index  [1] is not equilibrium index
	 [2] is not equilibrium index  [3] is not equilibrium index
	 [4] is not equilibrium index
findEquilibrium returns 2 since equilibrium found at index number 2.	findEquilibrium returns -1 since equilibrium not found at any index 0, 1, 2, 3 and 4.

```
int findEquilibrium(int arr[], int n)
{
    int Lsum, Rsum;
    for (int i = 0; i < n; ++i)
    {
        Lsum = 0;
        for (int j = 0; j < i; j++)
            Lsum += arr[j];
        Rsum = 0;
        for (int k = i + 1; k < n; k++)
            Rsum += arr[k];
        if (Lsum == Rsum)
            return i;
    }
    return -1;
}
```

Q5. [10] Morse code is an encoding scheme used in telecommunication to transmit text messages from one point to the other. The encoding scheme simply uses combinations of dots and dashes to represent alphabetic characters. We consider a fixed length code: each letter is represented by 5 symbols, as shown in the code table below.

A .-.-.	B -...-	C -.---.	D ---..	E .-..	F ..-..	G ----.	H-	I ..---	J .----	K --.-.	L .-..	M ---.-
N -.-..	O -----	P -.---	Q --.-.	R .-...	S ...--	T -....	U ..--.	V ...--.	W --.-.	X -.-..	Y -.---.	Z --..-

The code for space is Write a C++ function that can takes in a message in the form of fixed-length Morse code and converts it to English text. You need to decide the input parameters of the function. Sample example:

```
string code(char x)
```

```
{
```

```
    switch (x) {
```

```
    case 'A':
```

```
        return ".-.-.";
```

```
    case 'B':
```

```
        return "-...-";
```

```
    case 'C':
```

```
        return "-.---.";
```

```
    case 'D':
```

```
        return "---..";
```

```
    case 'E':
```

```
        return ".-..";
```

```
    case 'F':
```

```
        return "..-..";
```

```
    case 'G':
```

```
        return "----.";
```

```
    case 'H':
```

```
        return "....-";
```

```
    case 'I':
```

```
        return "..---
```